

Quiz 3: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 3 is Thursday Sep 17, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Read this first: the scope stops where the lecture stopped

Week 3 spent most of its time going back over the Week 2 fundamentals, partly because the recording was lost after the Tensors slide and partly because that material is worth the time. We then started backpropagation and got as far as “each node needs to know exactly two things,” and stopped there.

This quiz covers exactly that and nothing past it. If a topic is in the Week 3 deck but we did not reach it on Thursday, it is not on this quiz. It will be on Quiz 4.

On the quiz:

- Tensors: rank, and what the axes mean
- Depth versus width, and depth as repeated transformation
- Universal approximation, as interpolation versus extrapolation
- Finite differences: the recipe, the cost, and why it ends the argument
- Epoch, iteration, batch size, and what the learning rate does
- Why there is no analytical solution for a network’s weights
- The three ways to get a gradient
- Automatic differentiation, and that backpropagation is a **subset** of it
- Loss versus cost
- The chain rule, including the multivariable form
- Gradient notation, and the shape of a weight matrix’s gradient
- Computational graphs: acyclic, differentiable nodes, and the **two things each node needs to know**

Not on this quiz. All of it is Quiz 4:

- Working a backward pass through a graph and reading off the numbers
- Adding gradients where two paths meet
- Topological order and Kahn’s algorithm
- Reverse mode against forward mode
- The layer-level backward pass and its three gradients
- Jacobians, vanishing and exploding gradients
- Anything in PyTorch

Lab 3 is not on this quiz either, since it is now due Sep 23, after the quiz. Do the lab for Quiz 4.

Format. Same shape as Quiz 2: true/false, multiple choice and multiple answer for 20 of the 25 points, then two short written questions.

Part 1: The Week 2 material we went back over

Tensors

A **tensor** is an array with any number of axes. That is the whole definition, and it is the only data structure in deep learning.

Rank	Shape	What it is
0	()	a scalar
1	(d)	a vector, one feature vector
2	(N, d)	a matrix, a batch of vectors
3	(N, T, d)	a batch of sequences
4	(N, C, H, W)	a batch of images

The two examples from lecture are worth being able to explain. A **sequence** is rank 3 because each word is itself a vector of features, a sentence is a matrix of those, and a batch is a stack of sentences. An **image** is rank 4 because colour needs three channels, red, green and blue, each its own matrix, so a batch of images is rank 4.

Depth versus width

- **Width** is more units in a layer. Each unit is one more feature the layer can detect, so width raises the **dimensionality** of your representation.
- **Depth** is more layers. Each layer transforms the previous layer's output rather than the input, so depth **builds features out of other features**.

The lecture's framing: width gives you enough room to describe the data, and depth turns simple descriptions into complicated ones. Lines and edges become a nose, a nose becomes a face, a face becomes a particular person. The XOR figure is the small version of this: one ReLU layer moves the points until a straight line separates them, and the output layer is still only a straight line.

Universal approximation, as interpolation

The theorem says one hidden layer with enough units can approximate any continuous function on a bounded region. **The way to hold it is interpolation versus extrapolation.** Inside the range of data the network has seen, it does very well. Asked about inputs outside that range, it has no idea, because there is nothing intuitive in it; it is matrices being multiplied. Networks interpolate well and extrapolate badly.

Finite differences, and their cost

A derivative is a limit of differences, so approximate it directly:

$$\frac{\partial \mathcal{L}}{\partial w_j} \approx \frac{\mathcal{L}(w_j + \varepsilon) - \mathcal{L}(w_j)}{\varepsilon}$$

Nudge one parameter, re-run the forward pass, see how much the loss moved, put it back. Repeat for every parameter.

Know the cost. For n parameters one gradient costs $n + 1$ forward passes: one per parameter, plus one for the un-nudged baseline that every difference is measured against. **Forgetting the +1 is the most common error.**

Lab 2's network had 49 parameters, so one gradient was 50 forward passes and 400 steps came to about 20,000. It worked, and it is still hopeless: the cost scales with the parameter count, so a real model would need millions of forward passes per step. Deep learning is not possible this way, and not because the hardware is too slow.

Epoch, iteration, batch size

- **Batch size B :** the examples used for one update
- **Iteration:** one parameter update
- **Epoch:** one full pass through the training set

$$\text{iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil$$

The lecture example: 256 houses, batches of 32. Show the network 32 houses, it predicts all 32, you compare against the answers, it computes gradients for every parameter and updates once. That is one iteration. Work through all 256 and you have done one epoch, which is 8 iterations. You normally run many epochs.

Batch sizes are usually powers of two, and there is no deep reason; it fits memory tidily. The real constraint is how much fits on your hardware.

Learning rate

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \lambda \nabla \mathcal{L}$$

One number scales every parameter's step, which is why it matters most.

- **Too large:** you overshoot the minimum and ricochet across the valley, learning nothing, and eventually the loss becomes **nan**.
- **Too small:** the loss creeps and you settle into the first local minimum you meet with no momentum to escape it.

Both failures are worth recognizing from a loss curve. The cures, momentum and learning-rate schedules, are Week 4.

Part 2: Why backpropagation exists

There is no analytical solution

In a calculus class you differentiate a function, set the derivative to zero, and read off the minima and maxima. If you could do that with a loss function you would simply calculate the optimal weights and be finished.

You cannot, for two independent reasons, and **both are examinable**:

1. **Size.** A closed-form solution for a model with millions or billions of parameters is not obtainable. Think about how much work one substitution rule was in Calc II, then imagine 175 billion parameters.
2. **Non-convexity.** The loss surface is lumpy, with many local minima and many saddle points, so there is no single solvable global structure.

So we descend the surface instead of solving it, which means we need the gradient.

Method	Exact?	Cost for n parameters
--------	--------	-------------------------

Three ways to get a gradient

This is the slide to know cold, because it frames the whole week.

Method	Exact?	Cost for n parameters
Symbolic, by hand	yes	only feasible for tiny networks
Finite differences	no , approximate	$n + 1$ forward passes
Reverse-mode autodiff	yes	one forward pass and one backward pass

Finite differences are worse on **both** axes: approximate **and** slow.

Automatic differentiation and backpropagation

Automatic differentiation computes partial derivatives of an arbitrary computer program by applying the chain rule to every elementary operation the program ran. It works because every line of a forward pass is one of a short list: add, subtract, multiply, divide, exp, log, tanh, max. Every one has a derivative you already know.

Backpropagation is a subset of automatic differentiation. Autodiff is the general procedure, it predates deep learning, and it has other uses we do not care about here. Backprop is its application to neural networks. Know which one is the superset; that direction is the examinable part.

Loss and cost

- **Loss** ℓ_i : the error on a **single** example
- **Cost** $\mathcal{L}$: the average of those losses over a batch

The housing example: if a house is truly worth one million and you predicted 900,000, the loss for that house is based on the 100,000 error. Do that for all 32 houses in the batch and average, and you have the cost, a **single scalar** saying how far off the model is across the batch.

Part 3: The chain rule and computational graphs

The chain rule

A network is a composition, so the chain rule is the entire mathematical content of backpropagation:

$$\frac{d}{dx}[f(g(h(x)))] = f'(g(h(x))) g'(h(x)) h'(x)$$

One factor per stage. The worked example from lecture, with $f = e^x$, $g = \sin x$, $h = x^2$:

$$\frac{d}{dx}[e^{\sin(x^2)}] = e^{\sin(x^2)} \cdot \cos(x^2) \cdot 2x$$

The multivariable version is the one that matters. A parameter early in a network reaches the loss through more than one intermediate quantity, so the single-variable rule is not enough:

$$\frac{\partial \mathcal{L}}{\partial u} = \sum_k \frac{\partial \mathcal{L}}{\partial v_k} \frac{\partial v_k}{\partial u}$$

The generalized form extends to **any depth and any width**, which is the point: depth is more nesting, width is more parallel functions at each stage, and the same rule covers both.

Gradient notation

A **gradient** is a collection of partial derivatives. In $\nabla_{\mathbf{x}} F$, the subscript is what you differentiate with respect to and the big letter is what you differentiate. θ means the whole parameter set, $\mathcal{L}$ is the cost, and some sources write J instead.

So $\nabla_{\mathbf{W}} \mathcal{L}$ holds **one partial derivative per weight**, and it therefore has **the same shape as $\mathbf{W}$** . A 3×4 weight matrix has a 3×4 gradient. Both of those facts are quizzable.

Computational graphs

A neural network is a **directed acyclic graph**. Each node is a value that some operation produced; each edge is an operation with a known derivative.

Three properties from lecture:

- **Acyclic.** Nothing repeats or feeds back into itself. LSTMs have a cyclic flavour and we will meet them, but we still do not represent the graph with cycles, because backprop needs an order.
- **Every node must be differentiable.** Either the framework already knows the derivative of that operation, or you supply it. This is the requirement that makes the whole method work.
- **Nodes can be as primitive or as coarse as you like.** A sigmoid can be one node, or it can be expanded into the multiply, add, exponential and divide that make it up. Both are correct graphs of the same function.

The practical reading: this is how networks are actually represented in software. Each node is an object. Values flow forward along the edges, and gradients flow back along them.

The two things each node needs to know

This is where the lecture stopped, and it is the most likely written question.

Pick any node in any graph. To do its part of the backward pass it needs exactly two things:

1. **The gradient handed to it from downstream**, meaning how much the loss changes with respect to its own output. **It cannot work this out itself**, so it has to be given it from later in the network.
2. **The derivatives of its own inputs**, which it *can* work out alone, because it knows which operation it is. A multiply node knows how to differentiate a multiply.

Why this matters: it makes the network **modular**. All the interleaving and apparent complexity goes away. Every node does the same small job, the loss node starts the chain because it can differentiate itself, and each node hands its result back one step. That is the chain rule working from the outside in.

How a node actually receives that upstream gradient, and what happens at a node whose output feeds two places, is exactly where Week 4 begins.

Checklist

Before Thursday, confirm you can:

- ☐ Say what a tensor is, and why a batch of images is rank 4
- ☐ Say what width buys and what depth buys, in one sentence each
- ☐ Explain universal approximation as interpolation versus extrapolation
- ☐ Compute the forward passes one finite-difference gradient costs for n parameters, **including the baseline**
- ☐ Say why that cost ends the argument for real models
- ☐ Define epoch, iteration and batch size, and compute iterations per epoch
- ☐ Describe what too large and too small a learning rate each look like
- ☐ Give **two** reasons there is no analytical solution for a network's weights
- ☐ Name the three ways to get a gradient, with exactness and cost for each
- ☐ Say which of automatic differentiation and backpropagation is the superset
- ☐ Distinguish loss ℓ from cost $\mathcal{L}$
- ☐ State the chain rule, and say why the multivariable form is needed
- ☐ Say what $\nabla_{\mathbf{w}} \mathcal{L}$ contains and what shape it is
- ☐ Say why the graph must be acyclic and why every node must be differentiable
- ☐ **State the two things each node needs to know, and which one it cannot compute itself**
- ☐ Arrive with a calculator and nothing else; no notes are permitted